

Test Task: Frontend Developer

Public Holidays Explorer — Angular SPA

Estimated completion time: 16–24 hours

Overview

Build a single-page Angular application that allows users to explore public holidays from countries around the world. The app should fetch live data from a public REST API, display holidays in a clear and structured way, and provide a smooth, responsive user experience. This task is designed to evaluate your proficiency with Angular, component architecture, reactive programming, and attention to UX details.

Tech Stack

The following stack is mandatory. Do not substitute any of these tools.

Area	Required
Framework	Angular 17+
UI Component Library	PrimeNG or NGX-Bootstrap (your choice)
Language	TypeScript 5.x
Styling	SCSS
State Management	NgRx (Store + Effects)
Routing	Angular Router with lazy-loaded modules
Forms	Angular Reactive Forms
HTTP	Angular HttpClient with interceptors
Linting	ESLint + Prettier
Testing	Jest (unit tests required)

Features to Implement

1. Home Screen — Upcoming Holidays

The default landing screen must show upcoming public holidays from around the world.

- Fetch and display the next 10 upcoming holidays worldwide
- Each card shows: country flag, holiday name, local name, date, and country name
- Cards are sorted by date ascending
- A refresh button re-fetches the data
- Skeleton loading cards shown while data is loading

2. Country Search & Holiday Browser

A dedicated screen for browsing holidays by country and year.

- Autocomplete search field populated from the /AvailableCountries endpoint
- Year selector: current year $\pm$ 5 years, implemented as a segmented control or stepper
- Holiday list displays after country + year selection
- Each holiday row: date, local name, English name, type badge (Public / Bank / Optional)
- Filter bar to show only specific holiday types
- Export button to download the current list as a .csv file

3. Country Detail Page

Clicking a country from any screen navigates to a detail page showing:

- Country name, flag, and country code
- Total number of public holidays in the selected year
- A timeline view of holidays distributed across months
- The next upcoming holiday highlighted at the top
- Breadcrumb navigation back to the search screen

4. Favourites

Users can bookmark holidays for quick reference.

- Bookmark icon on each holiday card/row
- Favourites stored in localStorage
- Dedicated Favourites tab showing all saved holidays
- Ability to remove individual items or clear all

API

Use the free Nager.Date API. No API key or authentication is required.

Endpoint	Usage
GET /api/v3/AvailableCountries	Populate country selector
GET /api/v3/PublicHolidays/{year}/{countryCode}	Fetch holidays by country & year
GET /api/v3/NextPublicHolidaysWorldwide	Home screen upcoming holidays
GET /api/v3/CountryInfo/{countryCode}	Country detail page metadata

Base URL: <https://date.nager.at>

Architecture Requirements

NgRx State Management

- Global store must manage: countries list, holidays by country/year, favourites, loading/error states
- Side effects (HTTP calls) must go through NgRx Effects — no HTTP calls inside components
- Selectors must be used for all data access from templates
- Actions must be descriptive and follow the [Source] Event naming convention

Lazy Loading

- Each major section (Home, Search, Favourites) must be a lazily loaded feature module
- Shared components go into a SharedModule
- No feature module code should be imported directly in AppModule

HTTP Layer

- All HTTP calls must go through a dedicated HolidayService
- An HTTP interceptor must handle loading state (show/hide global spinner)
- An HTTP interceptor must handle error responses and dispatch error actions to the store

Code Quality

- ESLint must pass with zero warnings
- Prettier formatting must be enforced
- No any types — strict TypeScript throughout
- All interfaces defined in dedicated .model.ts files
- Unit tests required for: all NgRx reducers, all NgRx selectors, HolidayService
- Minimum test coverage: 60%

Evaluation Criteria

Criterion	Weight
Feature completeness	High
NgRx architecture correctness	High
TypeScript strictness & code quality	Medium
Error & loading state handling	Medium
Lazy loading & module structure	Medium
Unit test coverage	Medium
UI polish & responsiveness	Low
Git history quality	Low

Deliverable

- Public GitHub repository
- README with: local setup instructions, architecture decisions, known limitations
- All linting checks passing
- No console errors in the browser
- App runs with npm install && ng serve without additional configuration

Submissions that do not include NgRx or use a different state management approach will not be reviewed.
Submissions with a single commit will not be reviewed.